Informe de Resultados

Prueba de carga del servicio App Transaction Balance

Dato	Detalle
Elaborado por	Manuel Aycart · QA Engineer
Fecha del informe	13 de octubre de 2026
Fecha de la prueba	24 de abril de 2025, de 01:35 a 02:38 (≈ 63 minutos)
Herramienta de carga	k6 (resumen de fin de prueba)
Fuentes analizadas	textSummary.txt y gráfica de monitoreo VUs vs. http_reqs

1. Resumen ejecutivo

La prueba sometió al servicio a 140 usuarios virtuales (VUs) concurrentes durante unos 63 minutos: 276.650 peticiones con un promedio de 73,2 peticiones por segundo. Todos los umbrales (thresholds) configurados en el script aparecen como cumplidos, pero el análisis de la gráfica de monitoreo muestra un episodio de degradación de unos 7 minutos (01:51 a 01:58) en el que el throughput cayó de ~80 a ~14 peticiones por segundo con la misma carga. Los 6.759 errores (2,44 %) se concentran en la etapa sostenida y son en su mayoría HTTP 5xx.

> **Resultado: aprobada con observaciones críticas**
>
> La prueba cumple los umbrales configurados, pero no se recomienda certificar el servicio para esta carga hasta identificar y corregir la causa del episodio de degradación y verificar que no se repite.

Indicador	Valor	Lectura
Duración total	≈ 63 min (3.780 s)	1,5 min de subida, 60 min a 140 VUs y 1,3 min de bajada
Peticiones	276.650	Una petición por iteración
Throughput promedio	73,2 req/s	≈ 80,5 req/s en régimen estable
Usuarios virtuales	140 (máximo)	Constantes durante la etapa sostenida
Tiempo de respuesta	Mediana 613 ms · p95 1,57 s	Promedio 862 ms; máximo 29,93 s
Tasa de error	2,44 % (6.759)	88,6 % son HTTP 5xx; 99,96 % en la etapa sostenida
Checks exitosos	97,55 %	Cada check fallido corresponde a una respuesta HTTP con error

2. Alcance y metodología

Se analizó la transacción "App Transaction Balance" (consulta de saldo) a partir del resumen de fin de prueba de k6 y de la gráfica de monitoreo que relaciona los usuarios virtuales con las peticiones por segundo.

2.1 Perfil de carga reconstruido

Etapa	Horario	Duración	Usuarios virtuales
0 · Subida	01:35:15 – 01:36:48	≈ 1,5 min	0 → 140
1 · Sostenida	01:36:48 – 02:36:40	≈ 60 min	140
2 · Bajada	02:36:40 – 02:38:00	≈ 1,3 min	140 → 0

La duración total se confirma con el propio resumen: 276.650 peticiones / 73,18 req/s = 3.780 s. Los contadores y_failed_request_stage_0, _1 y _2 corresponden, por su numeración y por la forma de la gráfica, a estas tres etapas.

2.2 Modelo de carga y método de análisis

- Modelo cerrado: cada VU envía una petición, espera la respuesta y hace una pausa de ≈ 1 s (iteration_duration 1,86 s menos http_req_duration 0,86 s). Por eso el throughput depende del tiempo de respuesta.
- Se calcularon métricas derivadas a partir del resumen (fórmulas en el Anexo A) y se digitalizó la gráfica de monitoreo para estimar el throughput y los tiempos por tramo.
- Limitaciones: el resumen no muestra los valores de los thresholds (solo si se cumplieron), los valores leídos de la gráfica son aproximados y no se dispone de métricas del servidor.

3. Hallazgos

3.1 Relación entre usuarios virtuales y peticiones por segundo

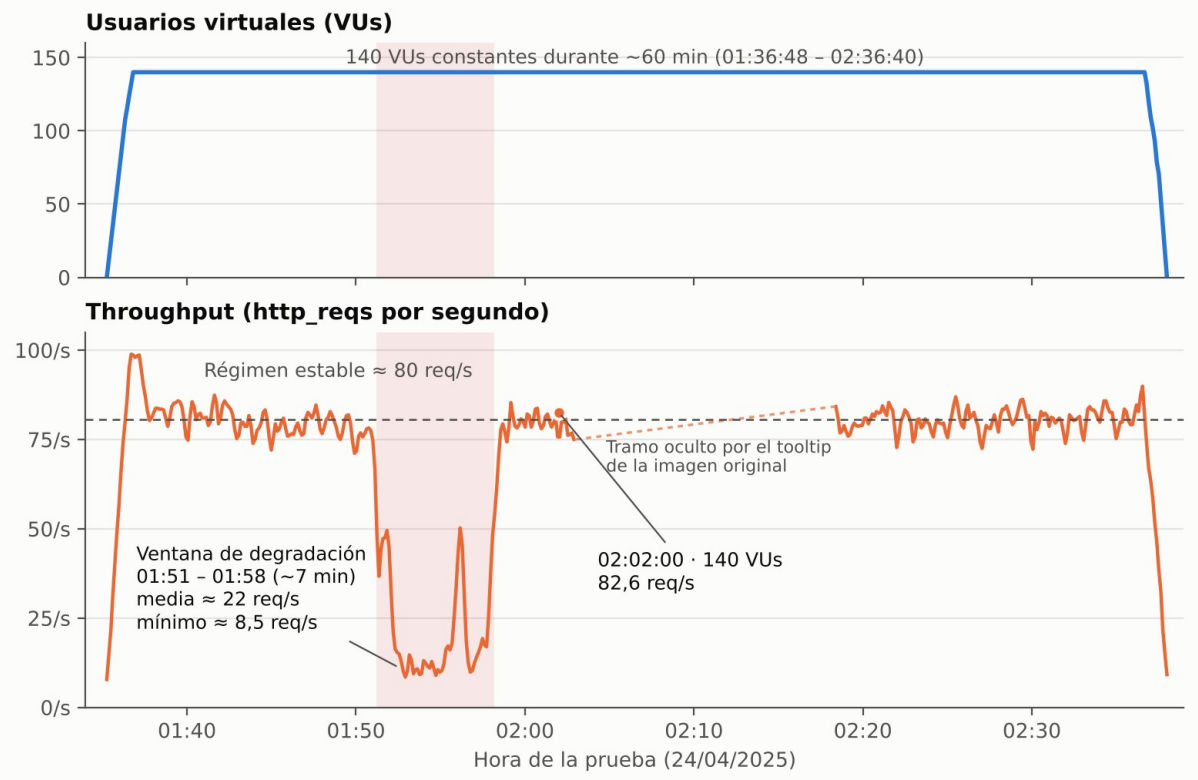


Figura 1. VUs y throughput durante la prueba, reconstruidos a partir de la gráfica de monitoreo (la gráfica original está en el Anexo B).

- **Régimen estable:** con 140 VUs constantes el throughput se mantuvo alrededor de 80,5 req/s (entre 71 y 99 req/s). Coincide con la relación del modelo cerrado: X = N / (R + Z) = 140 / (0,74 s + 1,0 s) ≈ 80 req/s.
- **Punto marcado a las 02:02:00:** 140 VUs y 82,6 req/s, lo que implica un tiempo de respuesta de ≈ 0,70 s en ese momento.
- **Ventana de degradación (01:51 a 01:58):** el throughput cayó a una media de 22 req/s, a ≈ 14 req/s en el tramo más profundo (01:52 a 01:58) y a un mínimo de ≈ 8,5 req/s, sin que cambiara el número de VUs. Es una caída de ~83 %. Aplicando la misma relación, el tiempo de respuesta subió a ≈ 9 s en promedio en el tramo más profundo y a ≈ 15 s en el mínimo.
- **Recuperación:** a las 01:56 hubo un intento de recuperación (~50 req/s) que volvió a caer; la recuperación definitiva llegó cerca de las 01:58 sin reducir la carga. Que el servicio se recupere solo apunta a un evento transitorio (pausas de recolección de basura, agotamiento del pool de conexiones, bloqueos en base de datos, un proceso batch o una dependencia externa) más que a un límite de capacidad permanente.
- **Resto de la prueba:** los otros 52 minutos de la etapa sostenida fueron estables, sin otra degradación visible.

Importante: en un modelo cerrado k6 reduce por sí mismo la carga cuando el sistema se pone lento, porque cada VU espera su respuesta antes de enviar la siguiente. La caída de throughput es un síntoma, no la causa. Con usuarios reales que siguen llegando, las peticiones se habrían acumulado y el impacto habría sido mayor.

3.2 Tiempos de respuesta

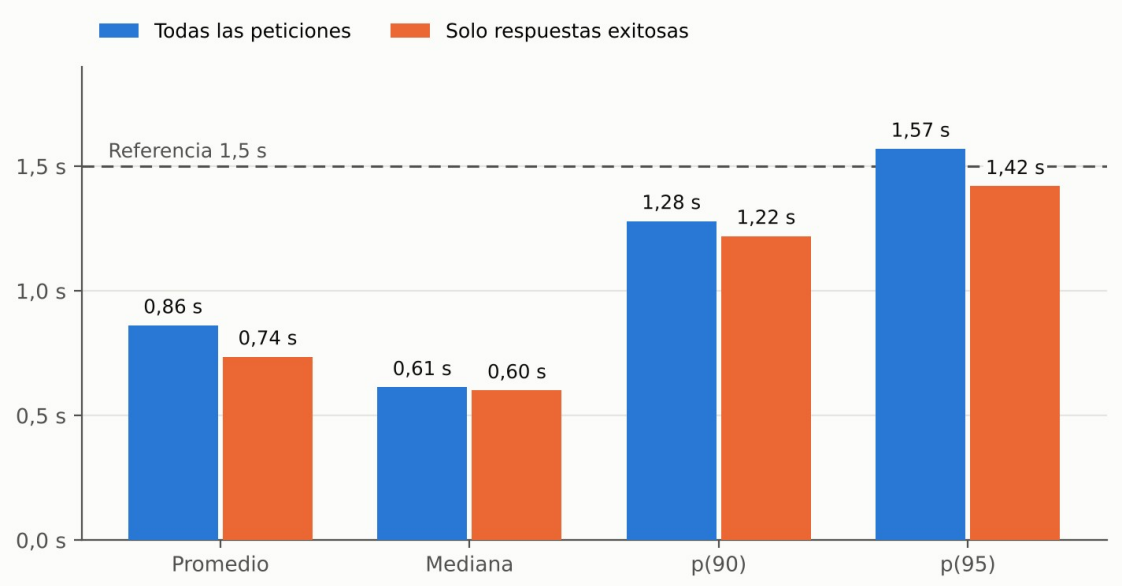


Figura 2. Estadísticos del tiempo de respuesta frente a una referencia de 1,5 s. Los máximos (29,93 s y 26,72 s) no se grafican para no aplanar la escala.

Métrica	Todas las peticiones	Solo respuestas exitosas
Promedio	861,68 ms	735,84 ms
Mínimo	191,86 ms	244,92 ms
Mediana	613,42 ms	600,70 ms
p(90)	1,28 s	1,22 s
p(95)	1,57 s	1,42 s
Máximo	29,93 s	26,72 s

- El promedio supera a la mediana en un 40 %: la distribución tiene una cola de respuestas lentas.
- **Las peticiones fallidas tardaron en promedio ≈ 5,9 s** (cálculo en el Anexo A). Los errores son lentos, típicos de timeouts, no rechazos inmediatos: el usuario espera varios segundos para luego recibir un error.
- El máximo de 29,93 s es consistente con un timeout de ~30 s en un componente intermedio (gateway o balanceador), por debajo del timeout por defecto de k6 (60 s).
- **El tiempo se consume en el servidor:** http_req_waiting (861,21 ms) representa el 99,9 % de http_req_duration. La red no es el cuello de botella: en promedio, bloqueo 10,97 µs, conexión 3,3 µs, TLS 7,36 µs, envío 43,22 µs y recepción 424,03 µs, con conexiones reutilizadas (keep-alive).
- Frente a un objetivo de p95 menor a 1,5 s (el mismo criterio del ejercicio 1), el p95 global de 1,57 s no cumpliría, mientras que el de las respuestas exitosas (1,42 s) sí: los errores lentos empujan el percentil por encima del límite.

3.3 Errores

Etapa	Tipo	Cantidad	% de los errores	Tasa en la prueba
0 · Subida	HTTP 5xx	1	0,01 %	0,0003 /s
1 · Sostenida	HTTP 4xx	769	11,38 %	0,20 /s
1 · Sostenida	HTTP 5xx	5.987	88,58 %	1,58 /s
2 · Bajada	HTTP 5xx	2	0,03 %	0,0005 /s
Total		6.759	100 %	2,44 % de 276.650

- **Las cifras cuadran entre sí:** checks fallidos (6.759) = http_req_failed (6.759) = failed_request (6.759) = suma de los contadores por etapa (1 + 769 + 5.987 + 2). Todos los checks fallidos corresponden a respuestas HTTP con error; no hubo respuestas exitosas con contenido incorrecto.
- **El 99,96 % de los errores ocurrió en la etapa sostenida.** Por su lentitud y por coincidir con la caída de throughput, lo más probable es que se concentren en la ventana de 01:51 a 01:58. En esa ventana se procesaron unas 9.300 peticiones; si la mayoría de los 6.756 errores de la etapa 1 ocurrió ahí, la tasa de error local habría rondado el 70 %. La tasa global de 2,44 % diluye el impacto real del incidente. Esta estimación debe confirmarse con la serie temporal de errores.
- **HTTP 5xx (88,6 %):** errores del servidor o de la infraestructura (500, 502, 503 o 504), consistentes con saturación y con timeouts del gateway.

- **HTTP 4xx (11,4 %):** no es habitual que aparezcan errores de cliente solo bajo carga. Hipótesis a validar: 429 (límite de peticiones), 401 o 403 (falla al validar el token cuando el servicio de autenticación está saturado) o 408 (timeout de la petición).

3.4 Tráfico de red

Se recibieron 842 MB (223 kB/s) y se enviaron 588 MB (156 kB/s), es decir ≈ 3,0 KB por respuesta y ≈ 2,1 KB por petición. El ancho de banda no es una limitación. El tamaño de cada petición sugiere cabeceras pesadas (por ejemplo, el token de autenticación), un punto a revisar como optimización.

3.5 Umbrales (thresholds)

Métrica con threshold	Estado reportado	Valor obtenido
checks	Cumplido (✓)	97,55 % exitosos
http_req_duration	Cumplido (✓)	p95 = 1,57 s
http_req_duration { expected_response:true }	Cumplido (✓)	p95 = 1,42 s
http_req_failed	Cumplido (✓)	2,44 %
http_reqs	Cumplido (✓)	73,18 req/s
iterations	Cumplido (✓)	73,18 iteraciones/s

- El check "App Transaction Balance response was OK" aparece marcado con ✗ porque tuvo fallos (97 % de éxito), aunque el threshold sobre la métrica checks se cumplió.
- Que todos los thresholds se cumplan a pesar de un incidente de 7 minutos muestra que evalúan promedios de toda la prueba y no detectan degradaciones acotadas en el tiempo.

4. Conclusiones

1. En régimen estable, el servicio sostiene ~80 req/s con 140 usuarios concurrentes, con una mediana de ~0,6 s y un p90 de ~1,2 s.
2. Hubo un episodio de degradación severa de unos 7 minutos (01:51 a 01:58) en el que el throughput cayó ~83 % y el tiempo de respuesta se multiplicó por más de 10, sin cambios en la carga. El servicio se recuperó solo, lo que indica un evento transitorio en el servidor o en sus dependencias.
3. Los 6.759 errores (2,44 %) se concentran en la etapa sostenida, son mayoritariamente 5xx y son lentos (≈ 5,9 s en promedio): los usuarios esperan varios segundos para recibir un error.
4. El cuello de botella está del lado del servidor: la espera de la respuesta representa el 99,9 % del tiempo total; la red y el generador de carga no influyen.
5. La prueba cumple los umbrales configurados, pero los promedios ocultan el incidente. Con la evidencia disponible, el servicio no debería certificarse para esta carga hasta conocer la causa raíz y comprobar que no se repite.
6. El modelo cerrado subestima el impacto: durante el incidente k6 inyectó menos carga porque cada VU esperaba su respuesta. Con usuarios reales que siguen llegando, la degradación habría sido mayor.

5. Recomendaciones

Prioridad	Acción	Objetivo
Alta	Correlacionar la ventana de 01:51 a 01:58 del 24/04/2025 con las métricas del servidor y del APM: CPU, memoria, pausas de GC, hilos, pool de conexiones a base de datos, consultas lentas o bloqueos, autoescalado, despliegues, procesos batch y latencia de dependencias.	Encontrar la causa raíz del episodio.
Alta	Desglosar los errores por código exacto (500, 502, 503, 504, 401, 429, etc.) etiquetando las peticiones en k6 y revisando los logs del gateway y de la aplicación.	Confirmar si los 5xx son timeouts e identificar los 4xx.
Alta	Revisar la cadena de timeouts (cliente, gateway y backend) e incorporar circuit breaker y reintentos con backoff para consultas idempotentes.	Fallar rápido y evitar esperas de hasta 30 s.
Media	Repetir la prueba para confirmar si el episodio es reproducible y ejecutar una prueba de resistencia de 2 a 4 horas.	Detectar si es periódico (jobs programados, fugas de memoria).
Media	Ejecutar con un modelo abierto (constant-arrival-rate) a la tasa objetivo, por ejemplo 80 TPS.	Medir el impacto real sin que k6 reduzca la carga.
Media	Ejecutar una prueba de estrés escalonada por encima de 140 VUs / 80 TPS.	Conocer la capacidad máxima y el margen disponible.
Media	Ajustar los thresholds al SLA del negocio: p(95) < 1,5 s sobre todas las respuestas, tasa de error < 1 %, umbrales por etapa o ventana de tiempo y abortOnFail.	Que la prueba falle cuando ocurra un incidente como este.
Baja	Enviar las métricas de k6 a Grafana (InfluxDB o Prometheus) junto con las del servidor.	Correlacionar en tiempo real.
Baja	Evaluar una caché de corta duración para la consulta de saldo, si el negocio lo permite, y reducir el tamaño de las peticiones.	Bajar la carga sobre el backend.

Anexo A. Cálculos

Cálculo	Fórmula y resultado
Duración de la prueba	276.650 / 73,176857 req/s = 3.780,6 s ≈ 63 min (también 769 / 0,203409 /s = 3.780,6 s)
Pausa por iteración (Z)	iteration_duration − http_req_duration = 1,86 s − 0,862 s ≈ 1,0 s
Ley de Little (modelo cerrado)	X = N / (R + Z), por lo tanto R = N / X − Z
R en régimen estable	140 / 80,5 − 1,0 ≈ 0,74 s
R a las 02:02:00	140 / 82,6 − 1,0 ≈ 0,70 s
R en el tramo más profundo	140 / 13,8 − 1,0 ≈ 9,1 s (mínimo: 140 / 8,5 − 1,0 ≈ 15,5 s)
Tiempo medio de las fallidas	(861,68 ms × 276.650 − 735,84 ms × 269.891) / 6.759 ≈ 5.887 ms
Bytes por petición	842 MB / 276.650 ≈ 3,04 KB recibidos; 588 MB / 276.650 ≈ 2,13 KB enviados

Cálculo	Fórmula y resultado
Peso del servidor en el tiempo	http_req_waiting / http_req_duration = 861,21 / 861,68 = 99,95 %
Caída de throughput	(80,5 − 13,8) / 80,5 ≈ 83 % en el tramo más profundo
Peticiones en la ventana	Integración de la gráfica digitalizada entre 01:51 y 01:58 ≈ 9.300

Anexo B. Fuentes originales

textSummary.txt

```
✗ App Transaction Balance response was OK
 ↳  97% — ✓ 269891 / ✗ 6759

 setup

✓ checks.........................: 97.55% ✓ 269891    ✗ 6759
  data_received..................: 842 MB 223 kB/s
  data_sent......................: 588 MB 156 kB/s
  failed_request.................: 2.44%  ✓ 6759      ✗ 269891
  http_req_blocked...............: avg=10.97µs  min=0s       med=0s       max=35.02ms p(90)=0s      p(95)=0s
  http_req_connecting............: avg=3.3µs    min=0s       med=0s       max=11.82ms p(90)=0s      p(95)=0s
✓ http_req_duration..............: avg=861.68ms min=191.86ms med=613.42ms max=29.93s  p(90)=1.28s   p(95)=1.57s
  ✓ { expected_response:true }...: avg=735.84ms min=244.92ms med=600.7ms  max=26.72s  p(90)=1.22s   p(95)=1.42s
✓ http_req_failed................: 2.44%  ✓ 6759      ✗ 269891
  http_req_receiving.............: avg=424.03µs min=0s       med=320.7µs  max=39.58ms p(90)=988.4µs p(95)=1.05ms
  http_req_sending...............: avg=43.22µs  min=0s       med=0s       max=31.25ms p(90)=0s      p(95)=517µs
  http_req_tls_handshaking.......: avg=7.36µs   min=0s       med=0s       max=27.02ms p(90)=0s      p(95)=0s
  http_req_waiting...............: avg=861.21ms min=191.86ms med=613.01ms max=29.93s  p(90)=1.28s   p(95)=1.57s
✓ http_reqs......................: 276650 73.176857/s
  iteration_duration.............: avg=1.86s    min=0s       med=1.61s    max=30.94s  p(90)=2.29s   p(95)=2.57s
✓ iterations.....................: 276650 73.176857/s
  vus............................: 2      min=2       max=140
  vus_max........................: 140    min=140     max=140
  y_failed_request_stage_0_HTTP5xx...: 1      0.000265/s
  y_failed_request_stage_1_HTTP4xx...: 769    0.203409/s
  y_failed_request_stage_1_HTTP5xx...: 5987   1.583625/s
  y_failed_request_stage_2_HTTP5xx...: 2      0.000529/s
```

Figura B1. Resumen de fin de prueba de k6 (textSummary.txt).

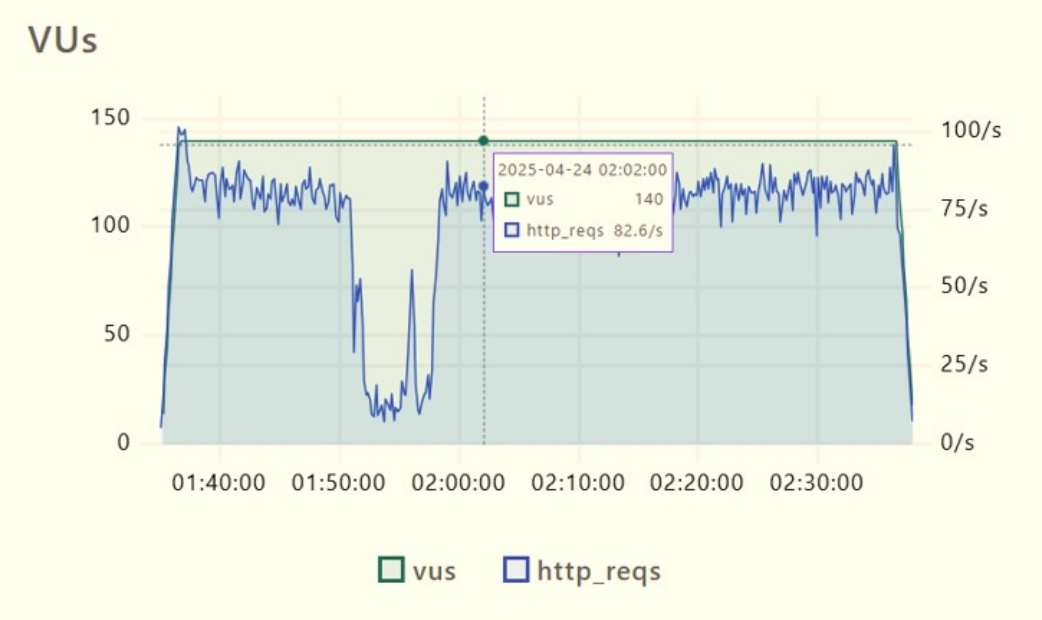


Figura B2. Gráfica de monitoreo original: VUs (área) y peticiones por segundo (línea).